

sensor. These functions calculate individual cell voltages using resistor divider values and derive temperature readings from the NTC thermistors. The collected data is processed to estimate key parameters such as voltage, current, temperature, and approximate SoC for each cell.

The processed data is then transmitted to the web dashboard, where it is displayed in a user-friendly format. The dashboard shows overall

battery pack voltage, current, and power, indicating whether the battery is charging or discharging. It also provides individual cell details, including voltage, temperature, SoC, and basic health status, enabling effective monitoring and diagnostics.

Fig. 3 shows the code snippet for Wi-Fi configuration, while Fig. 4 shows the code snippet that processes sensor data and calculates charge, current, and temperature

```

244 </script>
245 </body>
246 </html>
247 )rawliteral";
248
249 // ===== SENSOR READING =====
250 void readSensors() {
251   float maxV = 0, minV = 5.0;
252   float packV = 0;
253
254   for (int i = 0; i < NUM_CELLS; i++) {
255     cells[i].v = getCellVoltageCorrect(cellPins[i]);
256     cells[i].temp = getTemp(ntcPins[i]);
257     cells[i].soc = getSOC(cells[i].v);
258
259     if (cells[i].v > maxV) maxV = cells[i].v;
260     if (cells[i].v < minV) minV = cells[i].v;
261     packV += cells[i].v;
262   }
263
264   float current = ina219.getCurrent_mA();
265   float power = ina219.getPower_mW();
266   String health = getHealth(maxV - minV);
267
268   String json = "{\"cells\":[";
269   for (int i = 0; i < NUM_CELLS; i++) {
270     json += "{\"v\": " + String(cells[i].v,3) + ", \"temp\": " + String(cells[i].temp,1) + ", \"soc\": " + String(
271     if (i < NUM_CELLS-1) json += ",";
272   }
273   json += "], \"packV\": " + String(packV,2) +
274   ", \"current\": " + String(current,0) +
275   ", \"power\": " + String(power,0) +
276   ", \"health\": \"" + health + "\"}";
277   events.send(json.c_str(), "data", millis());
278 }
279
280

```

Fig. 4: Code snippet for calculating charge, current, and temperature from the IndusBoard COIN V2 I/O readings

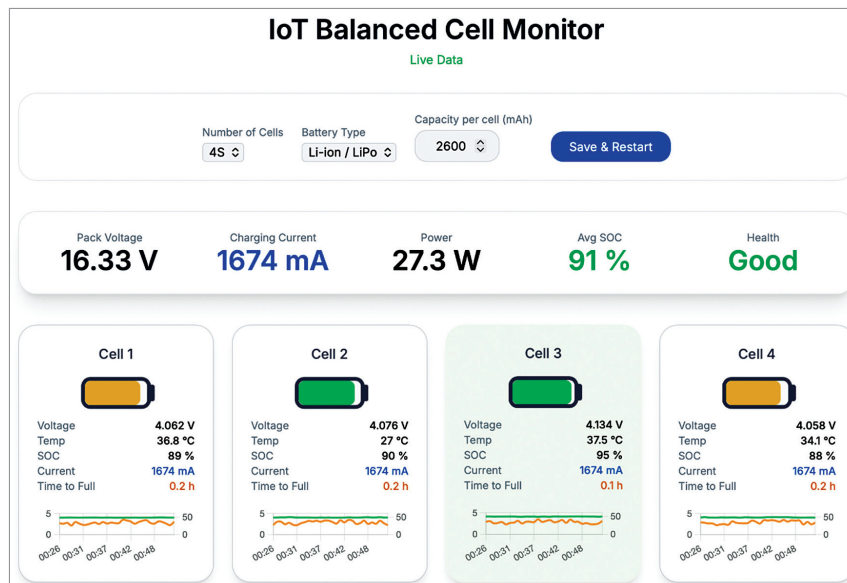


Fig. 5: IoT dashboard displaying the battery pack data

TO DOWNLOAD SOURCE CODE AND RESOURCES



Scan this QR code or go to <https://sourcecode.electronicshobby.com> to access and download source code and other resources for this project

from the readings obtained via the I/O pins of the IndusBoard COIN V2.

Construction and testing

First, upload the source code to the IndusBoard COIN V2, as shown in the author's prototype (Fig. 1). Power on the device, and it will attempt to connect to the configured Wi-Fi network. Once connected, the assigned IP address will be displayed on the serial monitor for dashboard access.

If the Wi-Fi connection is not established, the device automatically switches to AP mode. In this mode, the board's Wi-Fi network can be accessed directly using the predefined SSID and password. This ensures easy access to the system even without an external router.

After assembling the circuit as per the diagram, connect the battery cells and attach the NTC thermistors to each cell using thermal paste. Also connect the current sensor to the main terminal of the battery pack. Once everything is set, open a web browser on a mobile device or laptop connected to the same network, enter the IP address, and access the dashboard for real-time monitoring.

The dashboard allows configuration of the number of cells (1S to 4S or more), battery type (Li-ion, NiMH, etc), and cell capacity. It displays key parameters such as total battery voltage, current, power, individual cell temperature, and estimated SoC. Fig. 5 shows the IoT dashboard displaying real-time battery pack data. **EFY**

Ashwini Kumar Sinha, an IoT and AI enthusiast, is Tech Journalist at EFY.